

Finite State Automata:- (FSA):-

FSA is a computational model used to recognize patterns in input data. it is used in designing digital circuits, text processing, compilers and artificial intelligence.

A FSA is defined as 5-tuple:

$$FSA = (Q, \Sigma, \delta, q_0, F)$$

Q = Finite Set of States

Σ = Finite input alphabet.

δ = Transition function ($\delta: Q \times \Sigma \rightarrow Q$)

q_0 = Start State ($q_0 \in Q$)

F = Set of final (accepting) states ($F \subseteq Q$)

Ex:- Let's plug the values to formula.

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1, 2\}$$

q_0 = initial States $q_0 \in Q$

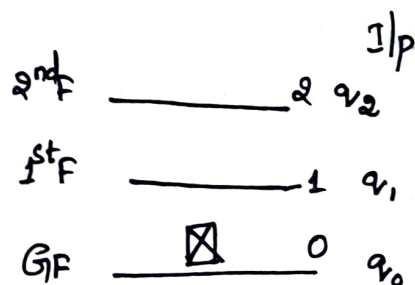
$$F = \{q_2\} \quad F \subseteq Q$$

$$s(q_0, 0) = q_0 \quad s(q_0, 1) = q_1, \quad s(q_0, 2) = q_2$$

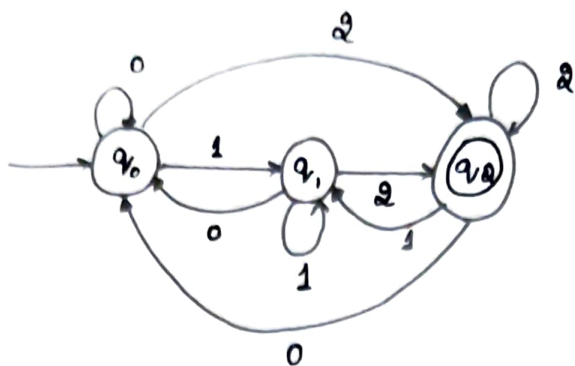
$$s(q_1, 0) = q_0 \quad s(q_1, 1) = q_1 \quad s(q_1, 2) = q_2$$

$$s(q_2, 0) = q_0 \quad s(q_2, 1) = q_1 \quad s(q_2, 2) = q_2$$

Let Ex:-



State diagram / Transition System :-



Transition Table :

Present state	Next state			← i/p's
	0	1	2	
→ q ₀	q ₀	q ₁	q ₂	
q ₁	q ₀	q ₁	q ₂	
q ₂	q ₀	q ₁	q ₂	

Types of Finite state Automata

① **Deterministic Finite Automata (DFA)**: Each state has exactly one transition per input symbol.

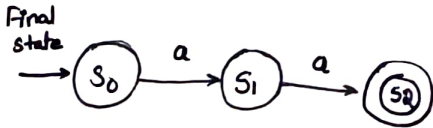
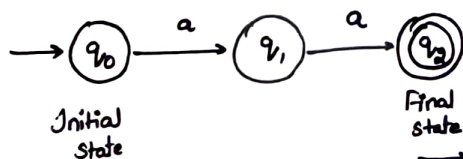
② **Non-deterministic Finite Automata (NFA)**: A state may have multiple transition for the same input symbol, including epsilon (ϵ) transitions.

Let us see DFA :-

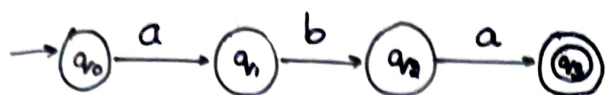
Deterministic :- Next state is known and has only one next state

Ex:- 1. $L_1 = \{aa\}$

Diff ways of writing Digm



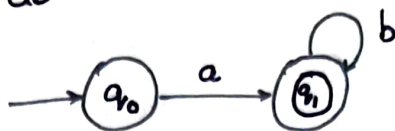
2. $L = \{aba\}$



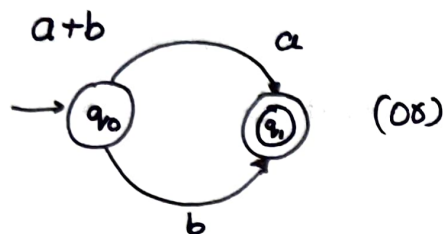
3. a^*



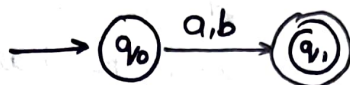
4. ab^*



5. $a \text{ or } b$



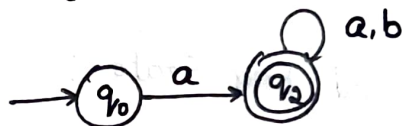
(OR)



6. $(a+b)^*$



7. $a(a+b)^*$



Defⁿ of DFA:-

It is a type of finite Automata that is used to define Languages in which next state is already known.

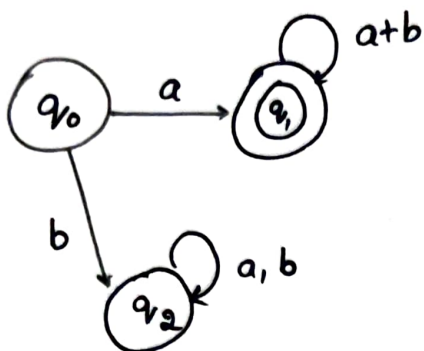
* only one output; Single letter cannot go to multiple state from one state.

Ex:- $R = a(a+b)^*$

Different States of DFA:-

Dead state, Trap state, Accepting state, Rejecting state, jump back

Ex:- $R = a(a+b)^*$



q_0 : initial state

q_1 : Final state

q_2 : Dead state (Dead End state)

Initial state:-

It is the state that Machine naturally starts in before it reads any input. it is called Entry point

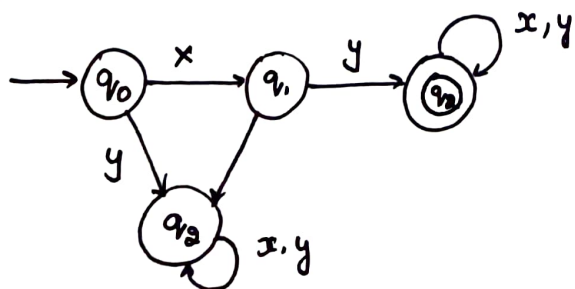
Final state:

It is the state where machine halts when it has no input left. it is called Accepting State.

Dead state / Dead End State:

it is also called as Rejecting state and Trap state. Once the Machine enters a dead state. there is no way for it to reach an accepting state

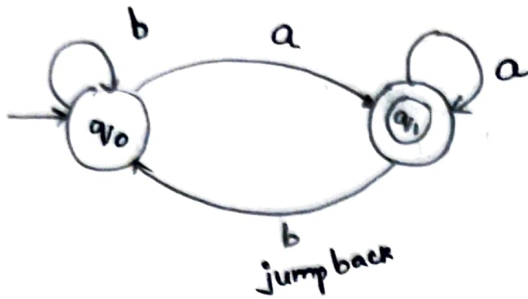
Ex:- Construct FA which recognizes the set of all strings defined over $\Sigma = \{x, y\}$ starting with the prefix 'xy'



$$R = (a+b)^*a$$

Possibilities:-

a
aa
aaa
ba
baa
bbaa
aba
abba
babbaa

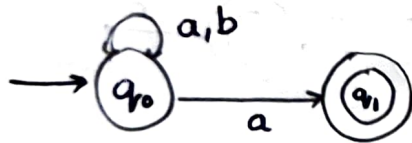


Non-Deterministic Finite Automata:-

NFA Rules :-

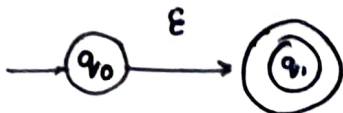
1. One letter can move to more than one state, next state is not determined.
2. Dead End State is not required.

Ex: $R = (a+b)^*a$



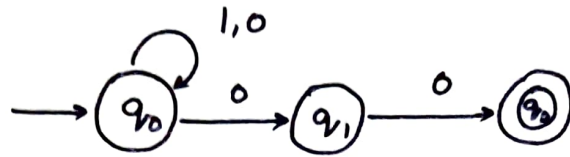
3. ϵ (epsilon) - NFA :

Can move to other state without any input. (i.e. it allows null/epsilon transition)



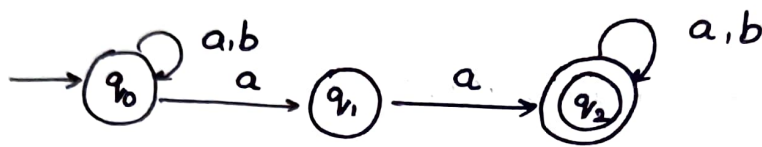
Ex: The Language $\{w \in \Sigma^* \mid w \text{ ends with } 00\}$ and $\Sigma = \{0, 1\}$

$$R = (1+0)^* 00$$



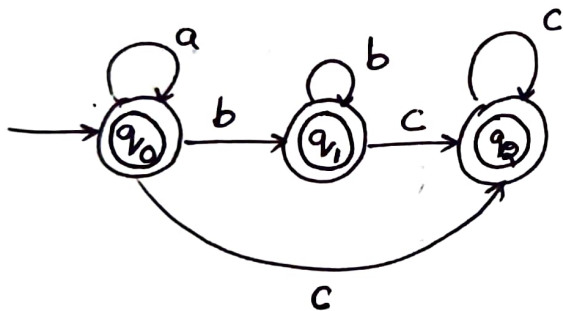
Ex: Accept all words that contains ^{Substring} aa over $\Sigma = \{a, b\}$

$$R = (a+b)^* aa (a+b)^*$$

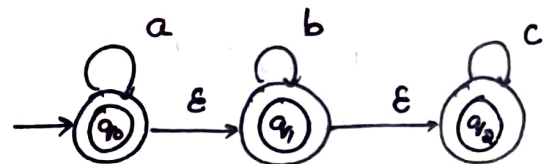


NFA with epsilon/ Null transition :-

$$Q = L = a^* b^* c^*$$



NFA with epsilon :-



All are final states because
accepts values from all